Movie Recommendation system using collaborative filtering

October 11, 2026

1 Assignment 1 - Collaborative Filtering Scaffold

1.1 1. Imports & Constants

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
RATINGS_DIR = r"ratings.csv"
MOVIES_DIR = r"movies.csv"

RANDOM_STATE = 42
```

1.2 2. Part A — Load & Explore

This section implements data loading, basic exploratory analysis, and pivots the user-item interaction matrix.

[2]:
```
def load_data(RATINGS_DIR=RATINGS_DIR, MOVIES_DIR=MOVIES_DIR):

    ratings_df = pd.read_csv(RATINGS_DIR)
    movies_df = pd.read_csv(MOVIES_DIR)
    print("Ratings DataFrame:")

    print(ratings_df.shape)
    ratings_df["timestamp"] = pd.to_datetime(ratings_df["timestamp"], unit='s')
    print(ratings_df.duplicated().sum())

    print("\n")

    print("Movies DataFrame:")
    print(movies_df.shape)
    print(movies_df.duplicated().sum())
    movies_df.drop_duplicates(inplace=True)
    ratings_df.drop_duplicates(inplace=True)
    ratings_df.dropna(inplace=True)
    movies_df.dropna(inplace=True)
```

```
    print("\n")
    return ratings_df, movies_df

def explore(ratings_df, movies_df):
    import matplotlib.pyplot as plt
    import seaborn as sns

    print("\n")

    print(ratings_df.head())
    ratings_df.info()
    ratings_df.describe()
    n_users = ratings_df['userId'].nunique()
    print(f"\n Number of unique users: {n_users}")
    n_movies = ratings_df['movieId'].nunique()
    print(f"\n Number of unique movies: {n_movies}")
    print(f"\n Number of duplicate ratings: {ratings_df.duplicated().sum()}")
    print(f"\n Number of duplicate movies: {movies_df.duplicated().sum()}")
    print(f"\n Number of missing ratings: {ratings_df['rating'].isnull().sum()}")
    print(f"\n Number of Zero ratings: {(ratings_df['rating']==0.0).sum().
 ↪sum()}")
    ratings=len(ratings_df)
    print(f"\n Number of ratings: {ratings}")
    Total_possible_ratings = n_users * n_movies
    sparsity = 1 - (ratings / Total_possible_ratings)
    print(f"\n Sparsity of the ratings matrix: {sparsity:.4%}")
    print("\n")
    print(movies_df.head())
    movies_df.info()
    movies_df.describe()


    # Count the occurrences of each rating and sort by the rating score index
    rating_counts = ratings_df['rating'].value_counts().sort_index()

    # Plot as a bar chart
    plt.figure(figsize=(8, 5))
    rating_counts.plot(kind='bar', color='skyblue', edgecolor='black')

    # Clean up the design
    plt.title('Distribution of Movie Ratings')
    plt.xlabel('Rating Score')
    plt.ylabel('Count')
    plt.xticks(rotation=0) # Keeps the x-axis numbers (1, 2, 3...) right-side up
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    plt.show()
```

```
    movie_summary = ratings_df.groupby('movieId')['rating'].agg(
        average_rating='mean',
        number_of_ratings='count'
    ).reset_index()
    movie_summary = movie_summary.merge(movies_df[["movieId", 'title']],␣
↪on='movieId')
    movie_summary = movie_summary[['movieId', 'title', 'average_rating',␣
↪'number_of_ratings']]
    print(movie_summary.head())

    sns.set_theme(style="whitegrid")
    plt.figure(figsize=(10, 6))

    sns.scatterplot(
        x='number_of_ratings',
        y='average_rating',
        data=movie_summary,
        alpha=0.5,
        color='teal',
        edgecolor=None
    )

    plt.title('Movie Quality vs. Total Number of Ratings (The Amazon Effect)',␣
↪fontsize=14, pad=15)
    plt.xlabel('Number of Ratings (Popularity)', fontsize=12)
    plt.ylabel('Average Rating Score (Out of 5 Stars)', fontsize=12)
    plt.show()


def build_matrix(ratings_df):
    return ratings_df.pivot(
        index='userId',
        columns='movieId',
        values='rating'
    )
```

1.3 3. Part B — Similarity Metrics

This section implements vector comparisons, index alignments, and the similarity precomputation cache.

[3]:
```
def get_co_rated(matrix, a, b, axis=0):
    if axis == 0:
        # Compare rows (users). a and b are user IDs.
        vec_a = matrix.loc[a]
        vec_b = matrix.loc[b]
```

```
    elif axis == 1:
        # Compare columns (items). a and b are movie IDs.
        vec_a = matrix[a]
        vec_b = matrix[b]
    else:
        raise ValueError("axis must be 0 (users/rows) or 1 (items/columns)")

    mask = vec_a.notna() & vec_b.notna()

    # Apply mask and convert to NumPy arrays
    return vec_a[mask].to_numpy(), vec_b[mask].to_numpy()

def euclidean_similarity(vec1, vec2):
    if len(vec1) < 2:
        return np.nan
    d = np.sqrt(np.sum((vec1 - vec2) ** 2))
    return 1.0 / (1.0 + d)

def cosine_similarity(vec1, vec2):
    if len(vec1) < 2:
        return np.nan
    norm1 = np.linalg.norm(vec1)
    norm2 = np.linalg.norm(vec2)
    if norm1 == 0 or norm2 == 0:
        return np.nan
    return np.dot(vec1, vec2) / (norm1 * norm2)

def pearson_similarity(vec1, vec2):
    if len(vec1) < 2:
        return np.nan
    v1_centered = vec1 - np.mean(vec1)
    v2_centered = vec2 - np.mean(vec2)
    return cosine_similarity(v1_centered, v2_centered)

# ===== BEGIN AI OPTIMIZATION (Claude) =====
# Faster "one target vs. everyone" similarity.
# It does the same job as the loop in _find_k_nearest_neighbors (get_co_rated ->␣
 ↪similarity_fn),
# but with plain NumPy instead of pandas, and it calls YOUR similarity functions,
# so the numbers are exactly the same as before:
#   1. Turn the matrix into a NumPy array (NaN = missing) and a True/False "was␣
 ↪rated?" array, once.
#   2. For a target, one NumPy comparison tells which other rows/columns share␣
 ↪>= 2 rated entries with it.
#   3. Call similarity_fn only on those; every other pair is undefined (NaN)␣
 ↪anyway.
```

```
_CACHE = {"matrix": None, "arrays": {}, "neighbors": {}}   # remembers work done␣
↪for ONE matrix

def clear_cache():
    """Forget everything cached. Call this if you edit a matrix in place."""
    _CACHE.update(matrix=None, arrays={}, neighbors={})

def _cache_for(matrix):
    # The cache belongs to one matrix; if a different matrix is passed in, start␣
↪fresh.
    if _CACHE["matrix"] is not matrix:
        _CACHE.update(matrix=matrix, arrays={}, neighbors={})
    return _CACHE

def _similarities_to_all(matrix, target, similarity_fn, axis=0):
    """Similarity of `target` to EVERY row (axis=0) or column (axis=1), as a␣
↪NumPy array
    (NaN = undefined). Same result as get_co_rated + similarity_fn for each pair.
↪"""
    arrays = _cache_for(matrix)["arrays"]
    if axis not in arrays:
        values = matrix.to_numpy(dtype=float)
        if axis == 1:
            values = np.ascontiguousarray(values.T)      # make the items the␣
↪rows
        arrays[axis] = (values, ~np.isnan(values))       # (ratings, "was rated?
↪")
    values, rated = arrays[axis]
    labels = matrix.index if axis == 0 else matrix.columns
    t = labels.get_loc(target)                           # raises KeyError for␣
↪an unknown target
    both = rated & rated[t]                              # True where the␣
↪candidate AND the target rated
    enough = np.where(np.count_nonzero(both, axis=1) >= 2)[0]   # >= 2 co-rated,␣
↪otherwise undefined
    sims = np.full(len(labels), np.nan)
    target_ratings = values[t]
    for i in enough:
        mask = both[i]
        sims[i] = similarity_fn(target_ratings[mask], values[i][mask])
    return sims
# ===== END AI OPTIMIZATION (Claude) =====

def precompute_similarity_matrix(matrix, similarity_fn, axis=0):
    if axis == 0:
        labels = matrix.index       # user IDs
```

```
    else:
        labels = matrix.columns      # movie IDs

    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    # Same table as the loop below: one row per label, using the faster helper above.
    # (The original pairwise loop below is kept for reference; it is no longer reached.)
    table = np.array([_similarities_to_all(matrix, a, similarity_fn, axis=0 if axis == 0 else 1) for a in labels])
    return pd.DataFrame(table, index=labels, columns=labels)
    # ===== END AI OPTIMIZATION (Claude) =====

    sim_df = pd.DataFrame(np.nan, index=labels, columns=labels)

    for i, a in enumerate(labels):
        for b in labels[i:]:
            vec1, vec2 = get_co_rated(matrix, a, b, axis=axis)
            score = similarity_fn(vec1, vec2)
            sim_df.loc[a, b] = score
            sim_df.loc[b, a] = score

    return sim_df
```

1.3.1 Section 3.2 Sanity Verification Harness

Run this cell once similarities are implemented to assert alignment correctness.

[4]:
```
print("Sanity verification harness ready...")

# ---- 1. Toy example from Section 3.2 -----------------------
# User A (generous): [5, 4, 5, 3]
# User B (harsh):    [3, 2, 3, 1]
vecA = np.array([5.0, 4.0, 5.0, 3.0])
vecB = np.array([3.0, 2.0, 3.0, 1.0])

# ---- 2. Euclidean similarity -------------------------------
d = np.sqrt(np.sum((vecA - vecB) ** 2))
euclid_sim = euclidean_similarity(vecA, vecB)
print(f"Euclidean distance    = {d:.4f}   (expected 4.0)")
print(f"Euclidean similarity  = {euclid_sim:.4f}   (expected 0.2)")
assert np.isclose(d, 4.0),           "Euclidean distance should be 4.0"
assert np.isclose(euclid_sim, 1/5),  "Euclidean similarity should be 1/(1+4)=0.2"

# ---- 3. Cosine similarity ----------------------------------
cos_sim = cosine_similarity(vecA, vecB)
print(f"Cosine similarity     = {cos_sim:.4f}   (expected ~0.987)")
```

```
assert np.isclose(cos_sim, 0.987, atol=1e-3), "Cosine should be ~0.987"

# ---- 4. Pearson similarity ---------------------------------
pearson_sim = pearson_similarity(vecA, vecB)
print(f"Pearson similarity    = {pearson_sim:.4f}   (expected 1.0)")
assert np.isclose(pearson_sim, 1.0, atol=1e-9), "Pearson should be exactly 1.0"

# ---- 5. get_co_rated on a tiny hand-made DataFrame ---------
toy_matrix = pd.DataFrame(
    {
        101: [5.0, 3.0, np.nan],
        102: [4.0, 2.0, np.nan],
        103: [5.0, 3.0, 1.0],
        104: [3.0, 1.0, np.nan],
        105: [np.nan, np.nan, 4.0],
    },
    index=["A", "B", "C"],
)

v1, v2 = get_co_rated(toy_matrix, "A", "B", axis=0)
print(f"get_co_rated(A,B,axis=0) -> {v1} , {v2}")
assert np.array_equal(v1, [5.0, 4.0, 5.0, 3.0])
assert np.array_equal(v2, [3.0, 2.0, 3.0, 1.0])

# A and C share only movie 103 -> only 1 co-rated -> undefined
v1, v2 = get_co_rated(toy_matrix, "A", "C", axis=0)
assert len(v1) == 1, "A and C only share 1 co-rated item"

# ---- 6. Undefined similarity cases -------------------------
# Fewer than 2 co-rated items -> np.nan
assert np.isnan(euclidean_similarity(np.array([5.0]), np.array([3.0])))
assert np.isnan(cosine_similarity(np.array([5.0]), np.array([3.0])))
assert np.isnan(pearson_similarity(np.array([5.0]), np.array([3.0])))

# Zero vector for cosine -> np.nan
assert np.isnan(cosine_similarity(np.array([0.0, 0.0]), np.array([1.0, 2.0])))

# Constant vector for Pearson -> np.nan (centered vector is all zeros)
assert np.isnan(pearson_similarity(np.array([5.0, 5.0, 5.0]),
                                   np.array([1.0, 2.0, 3.0])))

print("All sanity checks passed")
# ============================================================
# End of AI-generated section
# ============================================================
```

```
Sanity verification harness ready...
Euclidean distance    = 4.0000   (expected 4.0)
```

```
Euclidean similarity  = 0.2000   (expected 0.2)
Cosine similarity     = 0.9872   (expected ~0.987)
Pearson similarity    = 1.0000   (expected 1.0)
get_co_rated(A,B,axis=0) -> [5. 4. 5. 3.] , [3. 2. 3. 1.]
All sanity checks passed
```

1.4 KNN Algorithm

will be used for part C and part D

[5]:
```
# ===== BEGIN AI OPTIMIZATION (Claude) =====
# Neighbour cache. The ranked neighbour list of a target depends only on
# (matrix, metric, axis, target) -- NOT on k or on the item being predicted. So
 ↪it is
# computed once per target and every k / every test row reuses it
# (top-k is just the first k entries of the same sorted list, so the results are
 ↪identical).
_FAST_TOP = 50   # neighbours stored per target by default (largest k in the
 ↪assignment grid)

def _compute_neighbors(matrix, targets, similarity_fn, axis, top):
    store = _cache_for(matrix)["neighbors"]
    labels = matrix.index if axis == 0 else matrix.columns
    for target in targets:
        sims = _similarities_to_all(matrix, target, similarity_fn, axis)
        sims[labels.get_loc(target)] = np.nan                        # a target
 ↪is never its own neighbour
        pairs = [(labels[i], sims[i]) for i in np.where(~np.isnan(sims))[0]]   #
 ↪drop undefined scores
        pairs.sort(key=lambda x: x[1], reverse=True)                 # same sort
 ↪as the original loop
        store[(axis, similarity_fn, target)] = (pairs[:top], len(pairs) <= top)
 ↪ # (list, is it the full list?)

def _fast_neighbors(matrix, target, k, similarity_fn, axis):
    """Top-k [(neighbour_id, score), ...] from the cache (computed on first use).
 ↪"""
    axis = 0 if axis == 0 else 1
    store = _cache_for(matrix)["neighbors"]
    key = (axis, similarity_fn, target)
    entry = store.get(key)
    if entry is None or not (entry[1] or k <= len(entry[0])):        # not
 ↪cached, or cached list too short for this k
        _compute_neighbors(matrix, [target], similarity_fn, axis, max(k,
 ↪_FAST_TOP))
        entry = store[key]
    return entry[0][:k]
```

```
def _prefetch_neighbors(matrix, targets, similarity_fn, axis, top=_FAST_TOP):
    """Compute the neighbour lists for many targets up-front."""
    labels = matrix.index if axis == 0 else matrix.columns
    store = _cache_for(matrix)["neighbors"]
    targets = pd.unique(np.asarray(targets))
    targets = targets[pd.Index(targets).isin(labels)]       # unknown targets␣
 ↪are left to fail/skip as before
    todo = [t for t in targets if (axis, similarity_fn, t) not in store]
    _compute_neighbors(matrix, todo, similarity_fn, axis, top)

def _prefetch_for_test(matrix, test_df, similarity_fn, method, top=_FAST_TOP):
    if method == "user":
        _prefetch_neighbors(matrix, test_df["userId"].to_numpy(), similarity_fn,␣
 ↪0, top)
    else:
        _prefetch_neighbors(matrix, test_df["movieId"].to_numpy(),␣
 ↪similarity_fn, 1, top)
# ===== END AI OPTIMIZATION (Claude) =====


def _find_k_nearest_neighbors(matrix, target , k, similarity_fn ,axis=0␣
 ↪,get_co_rated_fn =None , sim_cache=None):
    """
    Finds the top k nearest neighbors for a target user or item.

    Args:
        matrix (pd.DataFrame): Utility matrix
        target (int): ID of target user or item
        k (int): Number of neighbors to fetch
        similarity_fn (function): Function to calculate similarity
        axis (int): Axis along which to find neighbors (0 for users, 1 for items)
        get_co_rated_fn (function): Function to get co-rated items/users
        sim_cache (dict): Cache for storing computed similarities

    Returns:
        list[tuple[int, float]]: List of (neighbor_id, similarity_score)
    """
    if get_co_rated_fn is None:
        get_co_rated_fn = get_co_rated

    #if the matrix is precomputed and cached, we can directly fetch the top k␣
 ↪neighbors from the cache
    #Ai generated
    if sim_cache is not None and target in sim_cache.index:
        row = sim_cache.loc[target].drop(labels=[target], errors="ignore").
 ↪dropna()
```

```
        return list(row.sort_values(ascending=False).head(k).items())
    #end of generation
    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    # Fast path: default co-rated function -> cached neighbour list built with␣
↪NumPy.
    # A custom get_co_rated_fn uses the original loop below.
    if get_co_rated_fn is get_co_rated:
        return _fast_neighbors(matrix, target, k, similarity_fn, axis)
    # ===== END AI OPTIMIZATION (Claude) =====
    if axis == 0:
        #searching users
        similares = matrix.index
    else:
        #searching items
        similares = matrix.columns
    neighbors=[]
    for similar in similares:
        if similar == target:
            continue

        v1,v2 =get_co_rated_fn(matrix ,target , similar, axis=axis)
        #ai generated
        if len(v1) < 2:
            continue
        #end of generation
        s =similarity_fn(v1,v2)
        if s is None or np.isnan(s):
            continue
        neighbors.append((similar,s))
        #the second element is the sort key and reverse 3la4an teb2a descending␣
↪order
    neighbors.sort(key= lambda x: x[1], reverse = True)
    return neighbors [:k]
```

1.5 4. Part C — User-based CF

This section implements user neighbor isolation and user-based prediction strategies.

[6]:
```
def find_k_nearest_users(matrix, target_user, k, similarity_fn,␣
↪get_co_rated_fn=None, sim_cache=None)-> list[ tuple[int , float]]:

    return _find_k_nearest_neighbors(matrix, target_user ,k , similarity_fn ,␣
↪axis =0, get_co_rated_fn=get_co_rated_fn, sim_cache=sim_cache)
```

```
def predict_rating_user_based(matrix, target_user, target_item, k,␣
↪similarity_fn, get_co_rated_fn=None, sim_cache=None):
    """
    Predicts rating for an item by an user based on neighbor user ratings.

    Args:
        matrix (pd.DataFrame): Utility matrix
        target_user (int): ID of target user
        target_item (int): ID of target movie/item
        k (int): Number of neighbors
        similarity_fn (function): Similarity metric
    Returns:
        float or None: Predicted rating, or None if prediction is unfeasible
    """
    #AI generated
    if target_item not in matrix.columns:
        return None # if the target item is not in the matrix, we cannot predict␣
↪its rating
    #end of generation
    neighbors = find_k_nearest_users(matrix ,target_user , k,␣
↪similarity_fn,get_co_rated_fn=get_co_rated_fn, sim_cache=sim_cache)
    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    item_ratings = matrix[target_item]   # single column lookup, reused for all␣
↪k neighbours
    # ===== END AI OPTIMIZATION (Claude) =====
    num =0.0
    denom= 0.0
    for neighbor, sim_score in neighbors:
        # ===== BEGIN AI OPTIMIZATION (Claude) =====
        # original: rating = matrix.loc[neighbor, target_item]
        rating = item_ratings.at[neighbor]
        # ===== END AI OPTIMIZATION (Claude) =====
        if pd.isna(rating):
            continue
        num= num + sim_score*rating
        denom+= abs(sim_score) #AI added abs() to avoid negative weights
    return None if denom == 0.0 else num / denom
```

1.6 5. Part D — Item-based CF

This section implements item-level recommendations and item-based prediction mechanisms.

[7]:
```
def find_k_nearest_items(matrix, target_item, k, similarity_fn,␣
↪get_co_rated_fn=None, sim_cache=None):
```

```
    return _find_k_nearest_neighbors(matrix,target_item,k, similarity_fn ,axis =␣
↪1, get_co_rated_fn=get_co_rated_fn, sim_cache=sim_cache)

def predict_rating_item_based(matrix, target_user, target_item, k, similarity_fn,
                              get_co_rated_fn=None, sim_cache=None):
    """
    Predicts rating for an item by a user based on item similarities.

    Returns:
        float or None: Predicted rating, or None if prediction is unfeasible
    """
    if target_user not in matrix.index:
        return None

    neighbors = find_k_nearest_items(matrix, target_item, k, similarity_fn,
                                     get_co_rated_fn=get_co_rated_fn,
                                     sim_cache=sim_cache)

    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    user_ratings = matrix.loc[target_user]    # single row lookup, reused for all␣
↪k neighbours
    # ===== END AI OPTIMIZATION (Claude) =====
    num, denom = 0.0, 0.0
    for neighbor_item, sim_score in neighbors:
        # ===== BEGIN AI OPTIMIZATION (Claude) =====
        # original: rating = matrix.loc[target_user, neighbor_item]
        rating = user_ratings.at[neighbor_item]
        # ===== END AI OPTIMIZATION (Claude) =====
        if pd.isna(rating):
            continue
        num += sim_score * rating
        denom += abs(sim_score)

    return None if denom == 0.0 else num / denom
```

1.7 6. Part E — Evaluation & Splits

Split operations and error diagnostics are handled interactively here.

1.7.1 Train/Test Splits

[ ]:
```
def train_test_split_ratings(ratings_df, test_fraction=0.2):
    df = ratings_df.sample(frac=1, random_state=42).reset_index(drop=True)

    df["rank"] = df.groupby("userId").cumcount()
    df["n"]    = df.groupby("userId")["userId"].transform("count")
```

```
    is_test = (df["rank"] >= df["n"] * (1 - test_fraction)) & (df["n"] >= 2)

    train_df = df.loc[~is_test, ["userId", "movieId", "rating"]].
↪reset_index(drop=True)
    test_df  = df.loc[ is_test, ["userId", "movieId", "rating"]].
↪reset_index(drop=True)

    return train_df, test_df
```

1.7.2 Metric Evaluator

[9]:
```
def evaluate(matrix, test_df, k, similarity_fn, method, get_co_rated_fn=None,␣
↪sim_cache=None):
    """
    Scores ratings predictions on test partition using RMSE, MAE and reports␣
↪un-predicted fractions.

    Args:
        matrix (pd.DataFrame): Training matrix
        test_df (pd.DataFrame): Test interaction DataFrame
        k (int): Neighbors
        similarity_fn (function): Distance metric
        method (str): 'user' or 'item' prediction process selection
    Returns:
        dict: {'rmse': float, 'mae': float, 'none_frac': float}
    """
    #Ai generated
    assert method in ("user", "item"), "method must be 'user' or 'item'"
    #fails if method not  "user" nor "item"
    #end of generation
    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    # Compute every needed neighbour list up-front in blocks (only when the fast␣
↪path applies).
    if sim_cache is None and (get_co_rated_fn is None or get_co_rated_fn is␣
↪get_co_rated):
        _prefetch_for_test(matrix, test_df, similarity_fn, method, max(k,␣
↪_FAST_TOP))
    # ===== END AI OPTIMIZATION (Claude) =====
    errors = []
    none_count = 0
    # ===== BEGIN AI OPTIMIZATION (Claude) =====
    # original:
    # for _, row in test_df.iterrows():
    #     u, i, true_r = int(row["userId"]), int(row["movieId"]),␣
↪float(row["rating"])
```

```
    # iterrows builds a pandas Series per row (slow); iterate over plain arrays␣
↪instead.
    for u, i, true_r in zip(test_df["userId"].to_numpy(), test_df["movieId"].
↪to_numpy(), test_df["rating"].to_numpy()):
        u, i, true_r = int(u), int(i), float(true_r)
    # ===== END AI OPTIMIZATION (Claude) =====

        if method == "user":
            pred = predict_rating_user_based(matrix, u, i, k, similarity_fn,
                                             get_co_rated_fn=get_co_rated_fn,
                                             sim_cache=sim_cache)
        else:
            pred = predict_rating_item_based(matrix, u, i, k, similarity_fn,
                                             get_co_rated_fn=get_co_rated_fn,
                                             sim_cache=sim_cache)
        if pred is None:
            none_count += 1
        else:
            errors.append(true_r - pred)

    errors = np.array(errors)
    n = len(errors)
    return {
        "rmse": float(np.sqrt(np.mean(errors ** 2))) if n else float("nan"),
        "mae":  float(np.mean(np.abs(errors))) if n else float("nan"),
        "n_predictions": n,
        "none_frac": none_count / len(test_df) if len(test_df) else float("nan"),
    }
```

1.7.3 Execution Grids (12 + 12 Configs)

- **(User-based)**: Executes User configs.
- **(Item-based)**: Executes Item configs.

[10]:
```
def run_grid_user_based(train_matrix, test_df):

    similarity_metrics = [
        ('Euclidean', euclidean_similarity),
        ('Cosine', cosine_similarity),
        ('Pearson', pearson_similarity)
    ]
    k_values = [5, 10, 20, 50]

    results = []

    user_col = 'userId'
    item_col = 'movieId'
```

```
    rating_col = 'rating'

    total_test_rows = len(test_df)

    for metric_name, sim_fn in similarity_metrics:
        # ===== BEGIN AI OPTIMIZATION (Claude) =====
        # neighbour lists do not depend on k: compute them once per metric for␣
↪all test users
        _prefetch_for_test(train_matrix, test_df, sim_fn, "user", max(k_values))
        # ===== END AI OPTIMIZATION (Claude) =====
        for k in k_values:
            predictions = []
            actuals = []

            # ===== BEGIN AI OPTIMIZATION (Claude) =====
            # original:
            # for _, row in test_df.iterrows():
            #     user = row[user_col]
            #     item = row[item_col]
            #     actual = row[rating_col]
            for user, item, actual in zip(test_df[user_col].to_numpy(),␣
↪test_df[item_col].to_numpy(), test_df[rating_col].to_numpy()):
            # ===== END AI OPTIMIZATION (Claude) =====

                if user not in train_matrix.index or item not in train_matrix.
↪columns:
                    continue

                pred = predict_rating_user_based(train_matrix, user, item, k,␣
↪sim_fn)

                if pred is not None and not np.isnan(pred):
                    predictions.append(pred)
                    actuals.append(actual)

            if len(predictions) > 0:
                preds_arr = np.array(predictions)
                acts_arr = np.array(actuals)
                rmse = np.sqrt(np.mean((preds_arr - acts_arr) ** 2))
                mae = np.mean(np.abs(preds_arr - acts_arr))
                none_frac = 1.0 - (len(predictions) / total_test_rows)
            else:
                rmse = np.nan
                mae = np.nan
                none_frac = 1.0

            results.append({
```

```
                'method': 'user_based',
                'similarity': metric_name,
                'k': k,
                'rmse': rmse,
                'mae': mae,
                'none_frac': none_frac
            })

    return pd.DataFrame(results)

def run_grid_item_based(train_matrix, test_df):
    """
    Evaluates Item-based predictions over all metrics and k values.
    Returns pd.DataFrame with columns: method, similarity, k, rmse, mae,␣
↪none_frac
    """
    similarity_metrics = [
        ('Euclidean', euclidean_similarity),
        ('Cosine',    cosine_similarity),
        ('Pearson',   pearson_similarity),
    ]
    k_values = [5, 10, 20, 50]
    results = []
    total_test_rows = len(test_df)

    for metric_name, sim_fn in similarity_metrics:
        # ===== BEGIN AI OPTIMIZATION (Claude) =====
        # neighbour lists do not depend on k: compute them once per metric for␣
↪all test items
        _prefetch_for_test(train_matrix, test_df, sim_fn, "item", max(k_values))
        # ===== END AI OPTIMIZATION (Claude) =====
        for k in k_values:
            predictions, actuals = [], []

            # ===== BEGIN AI OPTIMIZATION (Claude) =====
            # original:
            # for _, row in test_df.iterrows():
            #     user   = row['userId']
            #     item   = row['movieId']
            #     actual = row['rating']
            for user, item, actual in zip(test_df['userId'].to_numpy(),␣
↪test_df['movieId'].to_numpy(), test_df['rating'].to_numpy()):
            # ===== END AI OPTIMIZATION (Claude) =====

                if user not in train_matrix.index or item not in train_matrix.
↪columns:
                    continue
```

```
                pred = predict_rating_item_based(train_matrix, user, item, k,␣
↪sim_fn)

                if pred is not None and not np.isnan(pred):
                    predictions.append(pred)
                    actuals.append(actual)

            if len(predictions) > 0:
                preds_arr = np.array(predictions)
                acts_arr  = np.array(actuals)
                rmse = np.sqrt(np.mean((preds_arr - acts_arr) ** 2))
                mae  = np.mean(np.abs(preds_arr - acts_arr))
                none_frac = 1.0 - (len(predictions) / total_test_rows)
            else:
                rmse, mae, none_frac = np.nan, np.nan, 1.0

            results.append({
                'method': 'item_based',
                'similarity': metric_name,
                'k': k,
                'rmse': rmse,
                'mae': mae,
                'none_frac': none_frac,
            })

    return pd.DataFrame(results)
```

[ ]:
```
ratings_df, movies_df = load_data()
explore(ratings_df, movies_df)

train_df, test_df = train_test_split_ratings(ratings_df, 0.2)
print("train:", train_df.shape, "test:", test_df.shape)

train_matrix = build_matrix(train_df)
print("train_matrix:", train_matrix.shape)

user_results = run_grid_user_based(train_matrix, test_df)
item_results = run_grid_item_based(train_matrix, test_df)

display(user_results.sort_values("rmse").reset_index(drop=True))
display(item_results.sort_values("rmse").reset_index(drop=True))
```

```
Ratings DataFrame:
(100836, 4)
0
```

```
Movies DataFrame:
(9742, 3)
0
```

```
   userId  movieId  rating           timestamp
0       1        1     4.0 2000-07-30 18:45:03
1       1        3     4.0 2000-07-30 18:20:47
2       1        6     4.0 2000-07-30 18:37:04
3       1       47     5.0 2000-07-30 19:03:35
4       1       50     5.0 2000-07-30 18:48:51
<class 'pandas.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   userId     100836 non-null  int64
 1   movieId    100836 non-null  int64
 2   rating     100836 non-null  float64
 3   timestamp  100836 non-null  datetime64[s]
dtypes: datetime64[s](1), float64(1), int64(2)
memory usage: 3.1 MB

 Number of unique users: 610

 Number of unique movies: 9724

 Number of duplicate ratings: 0

 Number of duplicate movies: 0

 Number of missing ratings: 0

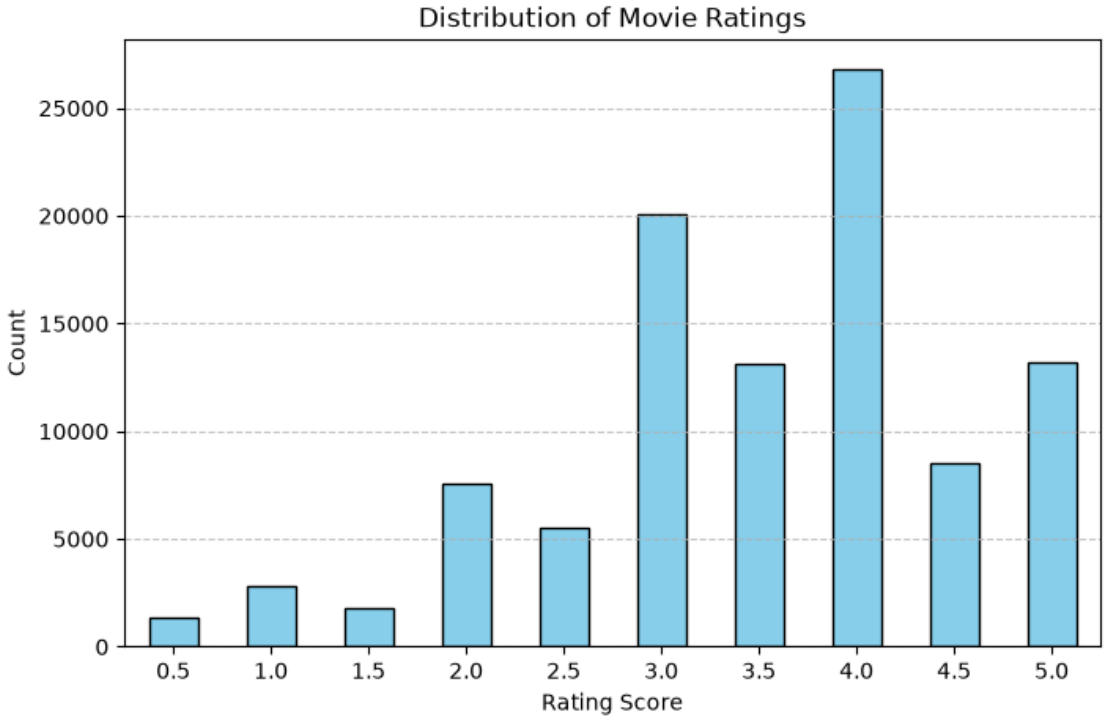
 Number of Zero ratings: 0

 Number of ratings: 100836

 Sparsity of the ratings matrix: 98.3000%
```

```
   movieId                                title  \
0        1                    Toy Story (1995)
1        2                      Jumanji (1995)
2        3             Grumpier Old Men (1995)
3        4            Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)
```

```
                                       genres
0  Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                              Comedy|Romance
3                        Comedy|Drama|Romance
4                                       Comedy
<class 'pandas.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   movieId  9742 non-null   int64
 1   title    9742 non-null   str
 2   genres   9742 non-null   str
dtypes: int64(1), str(2)
memory usage: 228.5 KB
```




```
   movieId                               title  average_rating  \
0        1                    Toy Story (1995)        3.920930
1        2                      Jumanji (1995)        3.431818
2        3             Grumpier Old Men (1995)        3.259615
3        4            Waiting to Exhale (1995)        2.357143
4        5  Father of the Bride Part II (1995)        3.071429
```

	number_of_ratings
0	215
1	110
2	52
3	7
4	49




```
train: (80896, 3) test: (19940, 3)
train_matrix: (610, 8947)
```

	method	similarity	k	rmse	mae	none_frac
0	user_based	Euclidean	50	1.110740	0.837098	0.611033
1	user_based	Cosine	50	1.121610	0.847313	0.562337
2	user_based	Euclidean	20	1.153435	0.861810	0.766851
3	user_based	Pearson	50	1.172323	0.887570	0.489619
4	user_based	Euclidean	10	1.184812	0.886350	0.850552
5	user_based	Euclidean	5	1.206002	0.890027	0.908776
6	user_based	Cosine	20	1.208963	0.909600	0.753460
7	user_based	Cosine	10	1.253954	0.937469	0.852156
8	user_based	Pearson	20	1.264805	0.953442	0.745085
9	user_based	Cosine	5	1.292703	0.967064	0.911184
10	user_based	Pearson	10	1.294080	0.970761	0.850251
11	user_based	Pearson	5	1.312045	0.985671	0.910582

	method	similarity	k	rmse	mae	none_frac
0	item_based	Euclidean	50	1.026672	0.772399	0.382648
1	item_based	Euclidean	20	1.063537	0.798622	0.517151
2	item_based	Cosine	50	1.097353	0.831625	0.392227
3	item_based	Euclidean	10	1.113936	0.837345	0.615697
4	item_based	Cosine	20	1.135205	0.859379	0.522718
5	item_based	Euclidean	5	1.145528	0.860054	0.716851
6	item_based	Cosine	10	1.168044	0.883436	0.625777
7	item_based	Cosine	5	1.196799	0.906821	0.724173
8	item_based	Pearson	20	1.248956	0.926771	0.488616
9	item_based	Pearson	50	1.258447	0.921765	0.370662
10	item_based	Pearson	10	1.261594	0.942328	0.583601
11	item_based	Pearson	5	1.267104	0.952181	0.689519

1.8 Part F — Analysis & Synthesis

1.8.1 Q12: Explanation on Reuse of Closest Elements

- **Core Approach:** The K-NN algorithm was reused by modifying the axis index between 0 and 1 to control and compute neighborhoods across either users or items respectively.

1.8.2 Q16: Performance Analysis of Configs

- **Best overall config:** Item-based, Euclidean, (k=50) → RMSE = 1.0267, MAE = 0.7724, none_frac = 0.3826.
- **Best user-based config:** User-based, Euclidean, (k=50) → RMSE = 1.1107, MAE = 0.8371, none_frac = 0.6110.
- **Effect of (k):** Increasing (k) from 5 to 50 consistently improves RMSE/MAE and reduces `none_frac` (more predictions).
- **Metric trend:** Euclidean generally performs best, then Cosine, then Pearson.
- **User vs Item CF:** Item-based CF outperforms user-based CF for most settings, with lower error and better coverage.

1.8.3 Q17: Metric Divergence Example (Euclidean vs Pearson)

A clear example is user-based CF at (k=50):

Metric	RMSE	MAE	none_frac
Euclidean	1.1107	0.8371	0.6110
Pearson	1.1723	0.8876	0.4896

- **Euclidean** gives lower error but predicts fewer ratings.
- **Pearson** predicts more ratings (lower `none_frac`) but has higher error.
- Reason: Euclidean measures absolute rating closeness, while Pearson centers each user's ratings and removes user bias. With sparse data, Pearson can be less stable because it needs enough co-rated items to estimate correlation reliably.

The same pattern appears in item-based CF: Euclidean (k=50) has RMSE = 1.0267 and none_frac = 0.3826, while Pearson (k=50) has RMSE = 1.2584 and none_frac = 0.3707.

1.8.4 Q18: Complexity and Scalability Analysis

1. Time Complexity

- **Similarity Matrix Precomputation:** Computing pairwise similarities across all users requires $O(U^2 \cdot M)$ in the worst case, where U is the total number of users and M is the total number of items. For each user pair, time is proportional to the number of co-rated items.
- **Inference / Prediction Time:** For every test interaction, searching for the top-k nearest neighbors takes $O(U)$ time. Across a test set of size T, the total evaluation grid search time scales as $O(\text{Configs} \cdot T \cdot U)$, where Configs accounts for the combination of similarity metrics and k values.

2. Space Complexity

- **Utility Matrix Storage:** A dense user-item matrix requires $O(U \cdot M)$ memory. Utilizing sparse matrix representations (such as SciPy's `csr_matrix`) reduces space complexity to $O(N)$, where N is the total number of non-zero rating interactions.
- **Similarity Cache:** Storing a precomputed user-user similarity matrix requires $O(U^2)$ memory, which can become a major memory bottleneck as the user base scales.

3. Scalability Bottlenecks

- **User Growth Vulnerability:** User-based collaborative filtering scales poorly when the number of users (U) vastly exceeds the number of items (M). As platforms grow, maintaining and updating a $U \times U$ similarity matrix becomes computationally prohibitive.
- **Sparsity Impact:** Real-world datasets exhibit high sparsity (often exceeding 99% empty entries). High sparsity reduces the overlap of co-rated items between users, resulting in undefined similarity scores (NaN) and increasing the unpredicted fraction (`none_frac`) during evaluation.

4. Mitigation & Architectural Optimizations

- **Caching Mechanisms:** Precomputing and caching similarity scores prevents redundant distance calculations during hyperparameter grid searches.
- **Item-Based Alternative:** In systems where users outnumber items ($U > M$), item-based collaborative filtering is often preferred because item relationships are more stable and item spaces change less frequently than user behavior.